

Vel Tech Group of Institutions

Name: GOPICHAND KAKIVAI
Email: vtu29748@veltech.edu.in
Roll no: vtu29748
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L3

VTU_DAA_Week 12_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 31

Section 1 : Coding

1. Problem Statement

You are given a graph, and your task is to find the minimum vertex cover for the given graph. A vertex cover is a subset of vertices in the graph such that every edge in the graph is incident to at least one vertex in the cover.

Input Format

The first line contains an integer 'vertices' representing the number of vertices in the graph.

The second line contains an integer 'edges' representing the number of edges in the graph.

The following 'edges' lines each contain two integers 'u' and 'v' representing an edge between vertices 'u' and 'v'.

Output Format

The output prints the minimum vertex cover for the given graph, where each vertex in the cover is separated by a space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7

6

0 1

0 2

1 3

3 4

4 5

5 6

Output: 0 1 3 4 5 6

Answer

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
int n, m;
```

```
int u[505], v[505];
```

```
int best[105], curr[105];
```

```
int bestSize;
```

```
bool isValid(int size) {
```

```
    bool covered[505] = {0};
```

```
    for(int i = 0; i < size; i++) {
```

```
        for(int j = 0; j < m; j++) {
```

```
            if(u[j] == curr[i] || v[j] == curr[i])
```

```
                covered[j] = true;
```

```
        }
```

```
    }
```

```
    for(int i = 0; i < m; i++)
```

```
        if(!covered[i]) return false;
```

```
    return true;
```

```

}

void solve(int idx, int size) {
    if(size >= bestSize) return;

    if(idx == n) {
        if(isValid(size)) {
            bestSize = size;
            for(int i = 0; i < size; i++)
                best[i] = curr[i];
        }
        return;
    }

    curr[size] = idx;
    solve(idx + 1, size + 1);

    solve(idx + 1, size);
}

int main() {
    scanf("%d", &n);
    scanf("%d", &m);

    for(int i = 0; i < m; i++)
        scanf("%d %d", &u[i], &v[i]);

    bestSize = n + 1;

    solve(0, 0);

    for(int i = 0; i < bestSize; i++)
        printf("%d ", best[i]);

    return 0;
}

```

Status : Wrong

Marks : 0/10

2. Problem Statement

A financial analyst is studying the daily profit and loss values of a company over a period of time. Each day's profit or loss is recorded as an integer in an array, where positive values indicate profit and negative values indicate loss.

The analyst wants to identify a continuous period of days during which the company achieved the maximum total profit. This period must consist of consecutive days only, and at least one day must be selected.

Given an array of integers representing daily profit and loss values, determine the maximum possible sum of any contiguous subarray.

To solve this efficiently, you must use a linear-time algorithm.

Input Format

The first line contains an integer n , representing the number of days.

The second line contains n space-separated integers representing daily profit or loss values.

Output Format

Print a single integer representing the maximum sum of a contiguous subarray.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 9
-2 1 -3 4 -1 2 1 -5 4
Output: 6

Answer

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);
```

```

int x;
scanf("%d", &x);

int max_sum = x, current = x;

for(int i = 1; i < n; i++) {
    scanf("%d", &x);
    if(current + x > x)
        current = current + x;
    else
        current = x;

    if(current > max_sum)
        max_sum = current;
}

printf("%d", max_sum);
return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Given an array of integers, determine whether there exists a majority element in the array.

A majority element is defined as an element that appears more than $n/2$ times in the array.

Your task is to implement a function `majorityElement()` that takes the input array and returns the majority element if it exists. If no such element exists, return -1.

Formula

An element is considered a majority element if:

$$\text{frequency}(\text{element}) > n/2$$

where n is the total number of elements in the array.

Input Format

The first line of input contains an integer n, representing the number of elements in the array.

The second line contains n space-separated integers a1 a2 a3 ... an representing the array elements.

Output Format

Print the majority element if it exists. Otherwise, print -1.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 9
3 3 4 2 4 4 2 4 4

Output: 4

Answer

```
#include <stdio.h>
```

```
int main() {  
    int n;  
    scanf("%d", &n);
```

```
  
    int arr[1005];  
    for(int i = 0; i < n; i++)  
        scanf("%d", &arr[i]);
```

```
  
    int candidate = 0, count = 0;
```

```
  
    for(int i = 0; i < n; i++) {  
        if(count == 0) {  
            candidate = arr[i];  
            count = 1;  
        } else if(arr[i] == candidate) {
```

```

        count++;
    } else {
        count--;
    }
}

count = 0;
for(int i = 0; i < n; i++)
    if(arr[i] == candidate)
        count++;

if(count > n / 2)
    printf("%d", candidate);
else
    printf("-1");

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Aarav is analyzing temperature fluctuations recorded over several days. The temperature changes are stored in an array of integers. He wants to identify a contiguous period of days such that the product of the temperature changes during that period is maximized.

Given an array of integers, determine the maximum product that can be obtained from any contiguous subarray containing at least one element.

Your task is to compute this value efficiently.

Formula

To correctly handle both positive and negative values, track both the maximum and minimum product ending at each index.

If the current element is negative, swap the previous maximum and minimum values before updating.

At each index i:

$\text{currentMax} = \max(\text{arr}[i], \text{arr}[i] \times \text{currentMax})$

$\text{currentMin} = \min(\text{arr}[i], \text{arr}[i] \times \text{currentMin})$

The final answer is the maximum value obtained among all currentMax values.

Input Format

The first line contains an integer n, representing the number of days.

The second line contains n space-separated integers representing the temperature change factors.

Output Format

The first line contains an integer n, representing the number of days.

The second line contains n space-separated integers representing the temperature change factors.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

2 3 -2 4 -1 5

Output: 240

Answer

```
#include <stdio.h>
```

```
int max(int a, int b) { return a > b ? a : b; }
```

```
int min(int a, int b) { return a < b ? a : b; }
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```



```

int arr[105];
for(int i = 0; i < n; i++)
    scanf("%d", &arr[i]);

int currMax = arr[0], currMin = arr[0], ans = arr[0];

for(int i = 1; i < n; i++) {
    if(arr[i] < 0) {
        int temp = currMax;
        currMax = currMin;
        currMin = temp;
    }

    currMax = max(arr[i], currMax * arr[i]);
    currMin = min(arr[i], currMin * arr[i]);

    ans = max(ans, currMax);
}

printf("%d", ans);
return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Sidhant is a logistics manager responsible for planning delivery routes for a fleet of trucks. He has a list of cities, and the distances between every pair of cities are given in the form of a matrix.

Each truck must start from a central warehouse (city 0), visit every other city exactly once, and return back to the warehouse. The goal is to minimize the total travel distance.

Sidhant needs to determine the minimum distance required to complete the route and also print the optimal path taken.

Input Format

The first line contains an integer n , representing the number of cities.

The next n lines contain n space-separated integers, representing the distance matrix.

Output Format

The first line of output prints the minimum travel distance.

In the next line, print the optimal path starting and ending at city 0.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0

Output: 80
0 1 3 2 0

Answer

```
#include <stdio.h>
#include <limits.h>
```

```
int n;
int dist[20][20];
int dp[1<<15][15];
int parent[1<<15][15];
```

```
int min(int a, int b) { return a < b ? a : b; }
```

```
int main() {
    scanf("%d", &n);
```

```
    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            scanf("%d", &dist[i][j]);
```

```
    int N = 1 << n;
```

```

for(int i = 0; i < N; i++)
    for(int j = 0; j < n; j++)
        dp[i][j] = INT_MAX;

dp[1][0] = 0;

for(int mask = 1; mask < N; mask++) {
    for(int u = 0; u < n; u++) {
        if(!(mask & (1 << u)) || dp[mask][u] == INT_MAX) continue;

        for(int v = 0; v < n; v++) {
            if(mask & (1 << v)) continue;

            int nextMask = mask | (1 << v);
            int newCost = dp[mask][u] + dist[u][v];

            if(newCost < dp[nextMask][v]) {
                dp[nextMask][v] = newCost;
                parent[nextMask][v] = u;
            }
        }
    }
}

```

```

int full = N - 1;
int ans = INT_MAX, last = -1;

for(int i = 1; i < n; i++) {
    int cost = dp[full][i] + dist[i][0];
    if(cost < ans) {
        ans = cost;
        last = i;
    }
}

```

```

printf("%d\n", ans);

```

```

int path[20], idx = 0;
int mask = full, curr = last;

while(curr != 0) {

```

```
    path[idx++] = curr;
    int temp = parent[mask][curr];
    mask ^= (1 << curr);
    curr = temp;
}

printf("0 ");
for(int i = idx - 1; i >= 0; i--)
    printf("%d ", path[i]);
printf("0");

return 0;
}
```

Status : Partially correct

Marks : 1/10